P2 Specification Document

Specification & Prototype / Week 7

This document describes the intended final scope for Lift & Aesthetics. It is a proposed baseline for review at P2; once approved, it becomes the specification used to assess the final delivery. The current prototype is a smaller working slice, and its actual capabilities are listed on page 8.

TECHNOLOGY STACK

C# / ASP.NET Core 10 MVC | Razor + HTML/CSS/JS | EF Core 10 | MySQL 8 | Stripe Sandbox

Money values are in RON. No live payment keys or real card charges are used.

PROJECT TEAM & RESPONSIBILITIES

Peca Ana Maria Cristina

FRONTEND & PRODUCT DISCOVERY

Catalog, search, responsive UI, Wishlist UI

Ploae Darius Adonis Ionuț

BACKEND, CART & PAYMENTS

Cart totals, checkout, Stripe, coupon rules

Nica David Andrei

DATABASE, ACCOUNTS & ADMIN

MySQL model, login, access rules, admin, tests

SCOPE RULE The three selected bonus proposals are B1, B5 and B7. Any scope change after P2 approval must follow the course approval process.

CHOSEN BONUS FEATURES

B1 FILTERED SEARCH

Combine category, price, availability and sorting; keep filters in the URL.

LEAD: Ana Maria (M1)

B5 DISCOUNT CODES

Percentage or fixed discounts with expiry and maximum-use checks on the server.

LEAD: Darius (M2)

B7 WISHLIST

Signed-in users save products and can move them to the cart.

LEAD: Ana Maria + David

Bonus features are planned scope, not claimed as complete in the current P2 prototype.

02 / REQUIREMENTS

Functional Requirements (A1-A12)

Each row describes the intended final behaviour. The prototype status is separated from the final implementation plan so the document does not imply unfinished features are already complete.

CODE	REQUIREMENT	IMPLEMENTATION PLAN	LEAD OWNER
A1	**Product catalog**	Paginated list (12-24 products per page), category pages and product detail views with name, description, price in RON, image and availability.	Ana (M1) + David (M3)
A2	**Basic search**	Search product names without case sensitivity; show matching products and a clear empty-state message when no result is found.	Ana (M1)
A3	**Shopping cart**	Add, update and remove lines. Recalculate prices and totals on the server, reject quantities above available stock and persist an authenticated user cart across logout.	Darius (M2)
A4	**User accounts**	Registration, login and logout with BCrypt password hashes; authenticated users can save and manage at least one delivery address.	David (M3)
A5	**Checkout**	Collect address, offer at least two delivery methods with different costs, show an itemised total and display a confirmation page with a unique order number.	Darius (M2) + Ana (M1 UI)
A6	**Sandbox payment**	Use Stripe test mode only. An order becomes Paid only after a valid asynchronous webhook; failed or abandoned payment stays Unpaid.	Darius (M2)
A7	**Stock management**	Decrement physical stock only after confirmed payment and restore it on cancellation. Active stock reservations and atomic checks prevent two customers claiming the last unit.	Darius (M2)
A8	**Order history**	Authenticated customers can view order date, products, quantities, total and current fulfilment status.	David (M3) + Ana (M1 UI)
A9	**Administration panel**	A separate Admin role can create, view, edit and delete products/categories, upload product images and filter orders by status; status changes follow the required workflow.	David (M3)
A10	**Basic security**	Validate inputs on the server, use anti-forgery protection and EF Core parameterised queries, and enforce Admin-only routes on the server.	David (M3)
A11	**Responsive design**	Customer-facing pages remain usable at 360 px width, with mobile-friendly navigation, product cards and checkout fields.	Ana (M1)
A12	**Automated tests**	Provide at least 10 tests, runnable with one command (`dotnet test`), covering cart calculations, stock rules and payment confirmation handling.	David (M3) + Darius (M2 tests)

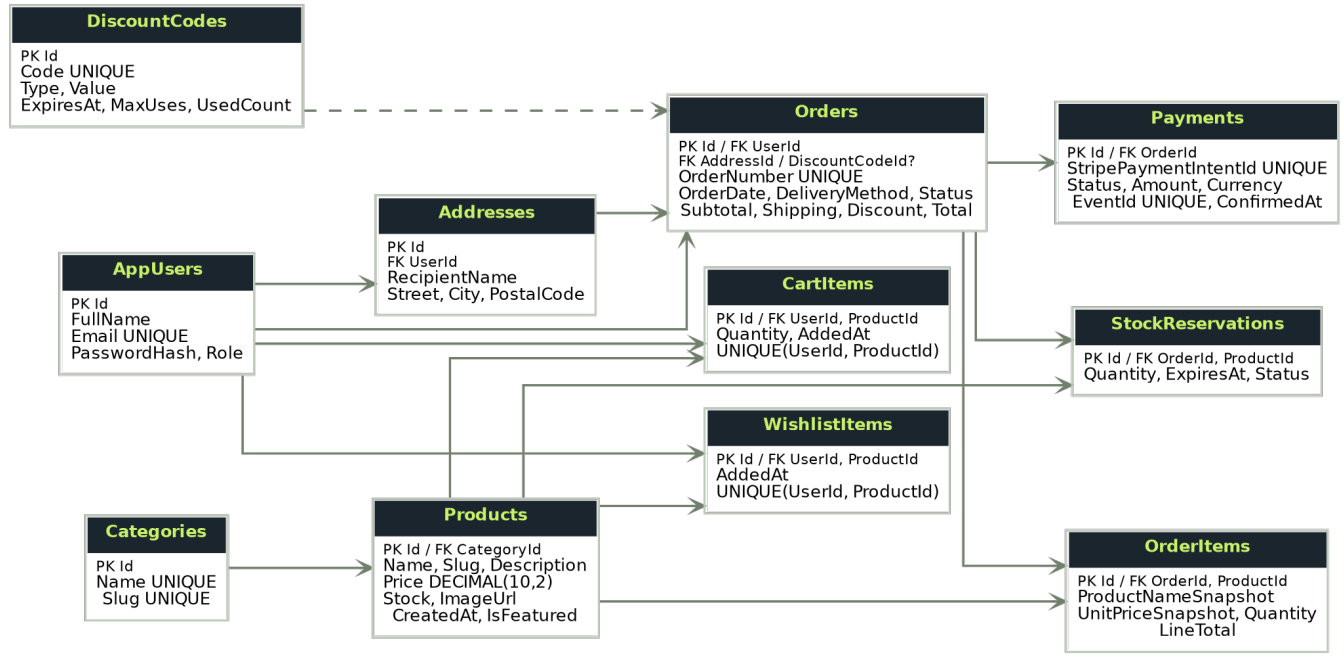
Important: the current P2 prototype covers a subset of A1/A2/A4. Full A1-A12 completion is the target for P3.

03 / MYSQL DATA MODEL

Data Model — Entity Relationship Diagram

Proposed final relational model for the agreed e-commerce scope. It replaces the unrelated membership / trainer entities in the previous draft and is designed around products, customers, orders, payment records and the selected bonuses.

PK = primary key | FK = foreign key | UNIQUE = unique constraint | arrows point from 1 to N (dashed = optional coupon link)



P2 PROTOTYPE VS FINAL MODEL

Currently implemented in the prototype: AppUsers, Categories and Products. The remaining entities shown above (cart, address, order, payment, discount, wishlist and stock reservation) are planned for the complete P3 application.

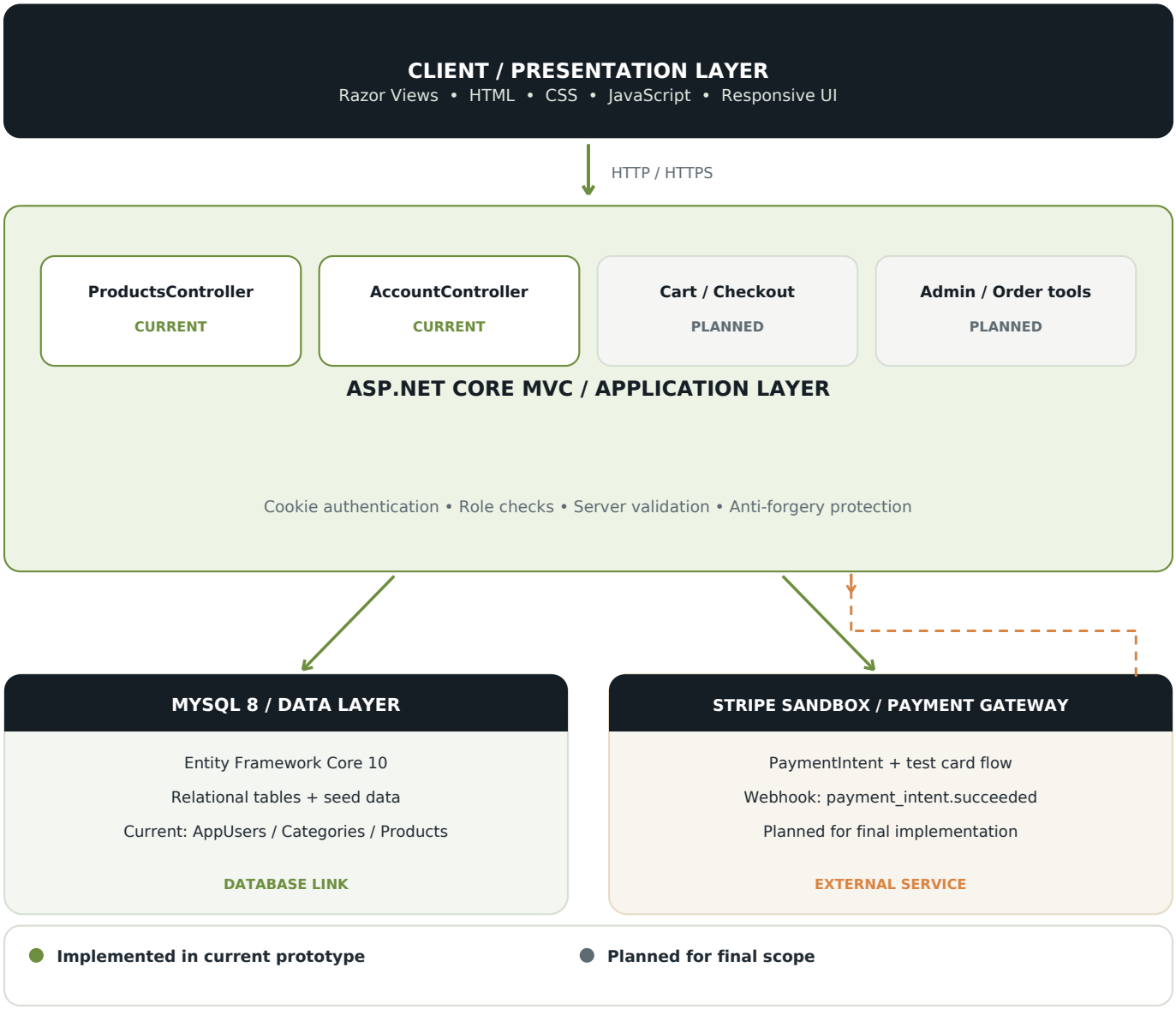
Relationship and data rules

- A user can have multiple addresses, cart lines, wishlist entries and orders; each product belongs to one category.
- Orders contain line-item price snapshots so later product price changes do not rewrite old receipts.
- StockReservations track temporary claims while payment is pending; confirmed payment decreases StockQuantity and completes the reservation.
- Store payment provider IDs and event IDs only; never store raw card numbers or security codes.

04 / APPLICATION ARCHITECTURE

System Architecture & Components

The target design uses ASP.NET Core MVC for request handling and business rules, EF Core for database access, MySQL for persistence, and Stripe Sandbox for the later payment integration.



Implementation boundaries

- Presentation: customer-facing pages use Razor, HTML, CSS and a small amount of JavaScript.
- Application: controllers call server-side logic; all prices, permissions, stock and payment status are validated on the server.
- Data: MySQL is the selected project database. EF Core maps C# models to relational tables.
- Payments: Stripe test mode is planned; the browser redirect alone will never mark an order as Paid.

05 / HTTP ROUTING & ACCESS LEVELS

Application Endpoints

Routes below describe the proposed final MVC/API surface. Rows tagged CURRENT are present in the P2 prototype; rows tagged PLANNED are included for the frozen final scope and must be implemented before P3.

CURRENT PLANNED

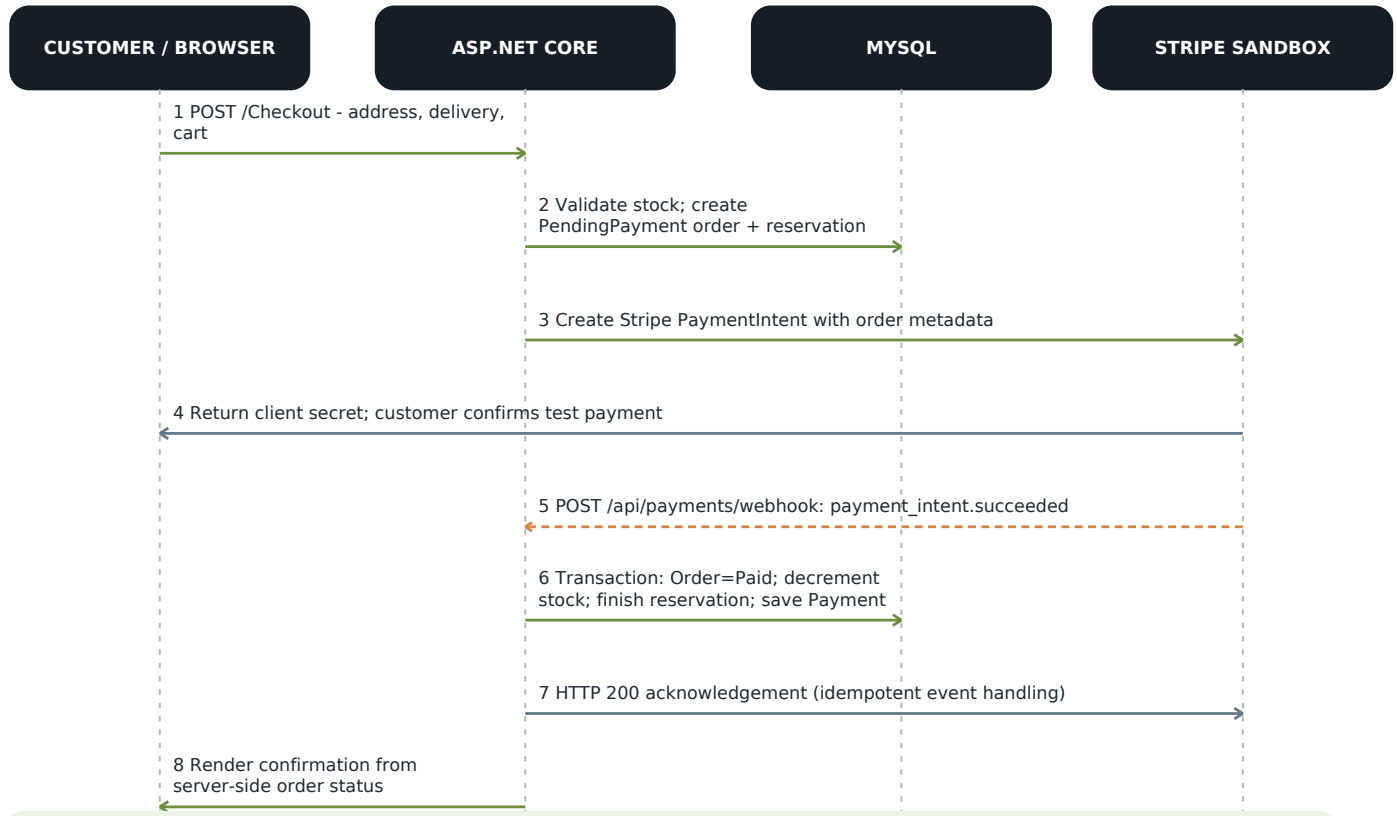
Access level is enforced on the server, not by hiding interface links.

METHOD	ROUTE	ACCESS	PURPOSE	STATE
GET	/Products	Public	Catalog with search, category filter and paging.	CURRENT
GET	/Products/Details/{id}	Public	Product details, price, stock and related items.	CURRENT
GET	/Account/Register	Public	Render account registration form.	CURRENT
POST	/Account/Register	Public	Validate input and create BCrypt password hash.	CURRENT
GET	/Account/Login	Public	Render sign-in form.	CURRENT
POST	/Account/Login	Public	Verify credentials and issue authentication cookie.	CURRENT
POST	/Account/Logout	Authenticated	End the current session.	CURRENT
GET	/Account/Index	Authenticated	Show account page for the signed-in user.	CURRENT
GET	/Cart	Authenticated	Show persisted cart lines, subtotal and available stock.	PLANNED
POST	/Cart/Add	Authenticated	Add a product after server-side stock validation.	PLANNED
POST	/Cart/UpdateQuantity	Authenticated	Update quantity and recalculate totals on server.	PLANNED
POST	/Cart/Remove	Authenticated	Remove a cart line.	PLANNED
POST	/Cart/ApplyCoupon	Authenticated	Validate B5 code, expiry and remaining uses.	PLANNED
GET	/Checkout	Authenticated	Show address, delivery methods and order summary.	PLANNED
POST	/Checkout	Authenticated	Create PendingPayment order and stock reservation.	PLANNED
POST	/api/payments/webhook	Stripe signature	Verify Stripe-Signature; update payment/order once.	PLANNED
GET	/Account/Orders	Authenticated	Show order history, dates, totals and statuses.	PLANNED
GET	/Account/Wishlist	Authenticated	Show saved items and move-to-cart action (B7).	PLANNED
GET	/Admin/Products	Admin role	View and filter product inventory.	PLANNED
POST	/Admin/Products/Create	Admin role	Create product and upload its image.	PLANNED
POST	/Admin/Products/Edit/{id}	Admin role	Update product details, price, image or stock.	PLANNED
POST	/Admin/Products/Delete/{id}	Admin role	Delete/deactivate a product after validation.	PLANNED

06 / SEQUENCE DIAGRAM

Critical Flow — Payment, Webhook & Stock

This sequence separates browser actions from trusted server-side payment confirmation. The webhook is the source of truth for marking an order Paid.



Webhook handler: verify the Stripe-Signature header with the server secret and ignore duplicate event IDs. Only a valid event updates payment and stock.

Rules for success, failure and concurrency

SUCCESS	Within one database transaction: mark the order Paid, decrement StockQuantity, complete the reservation and record the payment event.
FAILURE / EXPIRY	Keep the order Unpaid and release its active stock reservation. Do not decrement physical stock.
CANCELLATION	If an already-paid order is cancelled, restore its quantities exactly once and update the order status.
CONCURRENCY	Available stock is physical stock minus active reservations; reservation creation must be atomic so the last unit cannot be claimed twice.

07 / FIVE REQUIRED USER JOURNEYS

User Interface Wireframes

Conceptual wireframes for the target final UI. They focus on required controls and information rather than final artwork; the implementation must remain usable at 360 px width.

1. PRODUCT CATALOG — A1 / A2 / B1

Search products... | SEARCH

Category | Min price | Max price

☑ In stock only | Sort: price / newest

Whey Protein	Training Tee	Lifting Straps
129.99 RON	69.99 RON	34.99 RON
VIEW	VIEW	VIEW

♡ Save to Wishlist shown on each product card (B7)

2. PRODUCT DETAILS — A1 / B7

Product image

Core Whey Protein 1 kg

Supplements

129.99 RON

In stock: 24

Qty: 1 | ADD TO CART

♡ SAVE TO WISHLIST

Description, category, price and availability

Related products from the same category

3. SHOPPING CART — A3 / B5

PRODUCT	QTY	TOTAL
Whey Protein	1	129.99
Creatine	1	59.99

Coupon code | APPLY

CHECKOUT

Subtotal 189.98

Total + delivery

4. CHECKOUT & PAYMENT — A5 / A6

DELIVERY ADDRESS

Full name

Street / number

City / postal code

Delivery method

(●) Standard 15 RON

( ) Express 25 RON

ORDER SUMMARY

Items: 189.98

Delivery: 15.00

Total: 204.98 RON

Stripe test card

PAY IN TEST MODE

Confirmation after webhook

5. ADMINISTRATION PANEL — A9 / A10

PRODUCTS	ORDERS	LOW STOCK
32+	Pending	Alerts

PRODUCT	CATEGORY	PRICE / STOCK	ACTIONS
Core Whey Protein	Supplements	129.99 / 24	EDIT DELETE
Training T-shirt	Clothing	69.99 / 28	EDIT DELETE
Lifting Straps	Accessories	34.99 / 29	EDIT DELETE

Wireframes are targets. Current prototype covers catalog, product details and login; cart, checkout and admin screens remain planned.

08 / DEMO READINESS

Working Prototype & P2 Verification Guide

The P2 live demonstration must show a catalog whose seed products are read from MySQL and a working login. The checklist below is aligned with the current prototype source and separates implemented items from the remaining final-scope work.

CURRENT P2 PROTOTYPE

- 32 seeded products across 4 categories (MySQL)
- Catalog pagination, name search and category filter
- Product detail page with image, price and stock
- Register, login, logout and account page
- BCrypt password hashes; cookie-based login

PLANNED FOR FULL P3 SCOPE

- Cart, delivery addresses, checkout and order history
- Stripe webhook, stock reservations and order admin
- Complete B1 combined filters / URL preservation
- B5 server-side coupon validation and B7 saved wishlist
- Admin CRUD, image upload and 10+ automated tests

Suggested live demo sequence

STEP	ACTION	EXPECTED RESULT	CHECK
1	**Start app**	Run `docker compose up --build`; open `http://localhost:8080`. Confirm the container and database start.	To verify live
2	**Browse catalog**	Show products loaded from MySQL, 4 category labels, price and stock; point out pagination.	To verify live
3	**Search / filter**	Search `protein`, then select Supplements. Show a helpful result list.	To verify live
4	**Product details**	Open a product and explain the name, description, price, image and stock fields.	To verify live
5	**Login / register**	Sign in using the demo customer or create a new local account, then log out.	To verify live

Local demo accounts (from the current initializer)

Customer: demo@lift.local / LiftDemo123! **Admin role: admin@lift.local / AdminDemo123!**

Before submission

- Run the project locally and verify the shown demo steps on the actual presentation machine.
- Ensure the README lists the correct startup commands and declares AI tools used.
- Keep the ERD/endpoints as the final target model, but do not describe planned routes as implemented.
- Create the P2 commit/tag in the existing repository and submit the PDF and slide deck at least 24 hours before presenting.

Honesty rule for the demo: the current source does not yet include cart, Stripe payment, admin CRUD, persistent wishlist/coupons or the A12 test suite. Present these as planned work until implemented and verified.